

Singly Linked List Implementation in C

```
# include<stdio.h>
# include<malloc.h>

struct Node
{
    int val;
    struct Node *next;
};
struct Node *head,*temp;

void insertNodeAtLast(int ele){
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->next = NULL;
    newNode->val = ele;
    if(head==NULL){
        head = newNode;
    }else{
        temp = head;
        while (temp->next != NULL){
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

void insertNodeAtBegin(int ele){
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->next = NULL;
    newNode->val = ele;
    if(head==NULL){
        head = newNode;
    }else{
        newNode->next = head;
        head = newNode;
    }
}

int insertNodeAtSpec(int eletoInsert,int atInsert){
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->val = eletoInsert;
    struct Node * prev;
    if(head==NULL){
        head = newNode;
    }else{
        if (head->val==atInsert){
```

```

        newNode->next = head;
        head = newNode;
        return 1;
    }else{
        temp = head;
        while (temp->val != atInsert && temp!=NULL ){
            prev = temp;
            temp = temp->next ;
        }
        if(temp ==NULL){
            return 0;
        }else{
            prev->next = newNode;
            newNode->next = temp;
            return 1;
        }
    }
}

int deleteFirstNode(){
    if(head == NULL){
        return 0;
    }
    else if(head->next == NULL){
        head = NULL;
        free(head);
        return 1;
    }
    else{
        temp = head;
        head = head->next ;
        free(temp);
        return 1;
    }
}

int deleteLastNode(){
    struct Node *prev;
    if(head == NULL){
        return 0;
    }else{
        if (head->next==NULL){
            head = NULL;
            return 1;
        }else{
            temp = head;
            while (temp ->next != NULL){
                prev = temp;
            }

```

```

        temp = temp->next;
    }
    prev->next =NULL;
    free(temp);
    return 1;
}
}

int deleteSpecNode(int ele){
    struct Node *prev;
    if(head == NULL){
        return 0;
    }else{
        if(head->next==NULL && head->val==ele){
            head=NULL;
            free(head);
            return 1;
        }else if (head->next ==NULL && head->val != ele){
            return 0;
        }else if (head->val ==ele){
            temp = head;
            head = head->next;
            free(temp);
            return 1;
        }else{
            temp = head;
            while (temp ->val != ele && temp!=NULL){
                prev = temp;
                temp = temp->next;
            }
            prev->next = temp->next;
            free(temp);
            return 1;
        }
    }
}

int countNumber(){
    int count = 0;
    temp = head;
    if (head==NULL){
        return 0;
    }else{
        while (temp->next !=NULL){
            count++;
            temp = temp->next;
        }
        return count+1;
    }
}

```

```

    }
}

int traverse(){
    if(head==NULL){
        return 0;
    }else{
        temp=head;
        while(temp != NULL){
            printf("\t%d",temp->val);
            temp = temp->next;
        }
        return 1;
    }
}

int search(int ele){
    if(head==NULL){
        return 0;
    }else if (head->next==NULL && head->val ==ele){
        return 1;
    }else if (head->next==NULL && head->val != ele){
        return 0;
    }
    else{
        temp=head;
        while( temp!=NULL && temp ->val != ele ){
            temp = temp->next;
        }
        if(temp==NULL){
            return 0;
        }else{
            return 1;
        }
    }
}

int traverseReverse(struct Node *temp){
    if (temp==NULL){
        return 0;
    }
    traverseReverse(temp->next);
    printf("\t%d",temp->val);
}

void main(){
    int choicel;
    int choice2;
    int ele;

```

```

int atInsert;
int count;
do{
    printf("\n1.Insert an Element : ");
    printf("\n2.Delete An Element : ");
    printf("\n3.Traverse List : ");
    printf("\n4.Search Element : ");
    printf("\n5.Count of Element : ");
    printf("\n6.Reverse Traverse : ");
    printf("\n7.Exit : ");
    printf("\n\nEnter Your Choice : ");
    scanf("%d",&choice1);
    switch (choice1){
        case 1 :
            printf("\n1.Insert at begin : ");
            printf("\n2.insert at last : ");
            printf("\n3.insert at spec : ");
            printf("\n\nEnter Your Choice : ");
            scanf("%d",&choice2);
            switch (choice2){
                case 1:
                    printf("\nEnter the Element : ");
                    scanf("%d",&ele);
                    insertNodeAtBegin(ele);
                    break;
                case 2:
                    printf("\nEnter the Element : ");
                    scanf("%d",&ele);
                    insertNodeAtLast(ele);
                    break;
                case 3:
                    printf("\nEnter the Element to insert : ");
                    scanf("%d",&ele);
                    printf("\nElement at insert : ");
                    scanf("%d",&atInsert);
                    insertNodeAtSpec(ele,atInsert);
                    break;
            }
        break;
        case 2 :
            printf("\n1.delete at begin : ");
            printf("\n2.delete at last : ");
            printf("\n3.delete at spec : ");
            printf("\n\nEnter Your Choice : ");
            scanf("%d",&choice2);
            switch (choice2){
                case 1:
                    deleteFirstNode();
                    break;

```

```

        case 2:
            deleteLastNode();
            break;
        case 3:
            printf("\nEnter the Element to Delete : ");
            scanf("%d",&ele);
            deleteSpecNode(ele);
            break;
    }
    break;
    case 3:
        printf("\n\n====Content of the List====\n\n");
        if(!traverse()){
            printf("List is Empty : ");
        }
        break;
    case 4:
        printf("\nEnter the Element to Search : ");
        scanf("%d",&ele);
        if(search(ele)){
            printf("\nElement Found\n");
        }else{
            printf("\nElement Not Found\n");
        }
        break;
    case 5:
        count = countNumber();
        printf("\nNUmber of the Nodes %d: \n",count);
        break;
    case 6:
        if (head==NULL){
            printf("\n\n===List is Empty==\n\n");
        }else{
            printf("\n\n===Reverse Content of the
List===\n\n");
            traverseReverse(head);
        }
    }
    }while (choice1 != 7);
}

```

Output:

```
1.Insert an Element :  
2.Delete An Element :  
3.Traverse List :  
4.Search Element :  
5.Count of Element :  
6.Reverse Traverse :  
7.Exit :
```

```
Enter Your Choice : 1
```

```
1.Insert at begin :  
2.inset at last :  
3.insert at spec :
```

```
Enter Yout Choice : 1
```

```
Enter the Element : 10
```

```
1.Insert an Element :  
2.Delete An Element :  
3.Traverse List :  
4.Search Element :  
5.Count of Element :  
6.Reverse Traverse :  
7.Exit :
```

```
1.Insert an Element :  
2.Delete An Element :  
3.Traverse List :  
4.Search Element :  
5.Count of Element :  
6.Reverse Traverse :  
7.Exit :
```

```
Enter Your Choice : 1
```

```
1.Insert at begin :  
2.inset at last :  
3.insert at spec :
```

```
Enter Yout Choice : 2
```

```
Enter the Element : 20
```

1.Insert an Element :
2.Delete An Element :
3.Traverse List :
4.Search Element :
5.Count of Element :
6.Reverse Traverse :
7.Exit :

Enter Your Choice : 1

1.Insert at begin :
2.inset at last :
3.insert at spec :

Enter Yout Choice : 2

Enter the Element : 40

1.Insert an Element :
2.Delete An Element :
3.Traverse List :
4.Search Element :
5.Count of Element :
6.Reverse Traverse :
7.Exit :

Enter Your Choice : 1

1.Insert at begin :
2.inset at last :
3.insert at spec :

Enter Yout Choice : 3

Enter the Element to insert : 30

lement at insert : 40

1.Insert an Element :
2.Delete An Element :
3.Traverse List :
4.Search Element :
5.Count of Element :
6.Reverse Traverse :
7.Exit :

Enter Your Choice : 3

====Content of the List====

10 20 30 40

```
1.Insert an Element :
2.Delete An Element :
3.Traverse List :
4.Search Ellement :
5.Count of Element :
6.Reverse Traverse :
7.Exit :
```

Enter Your Choice : 6

===Reverse Content of the List===

40 30 20 10

```
1.Insert an Element :
2.Delete An Element :
3.Traverse List :
4.Search Ellement :
5.Count of Element :
6.Reverse Traverse :
7.Exit :
```

Enter Your Choice : 5

NUmber of the Nodes 4:

```
1.Insert an Element :
2.Delete An Element :
3.Traverse List :
4.Search Ellement :
5.Count of Element :
6.Reverse Traverse :
7.Exit :
```

Enter Your Choice : 4

Enter the Element to Search : 50

Element Not Found

=====Content of the List=====

```
10      20      30      40
1.Insert an Element :
2.Delete An Element :
3.Traverse List :
4.Search Ellement :
5.Count of Element :
6.Reverse Traverse :
7.Exit :
```

Enter Your Choice : 2

```
1.delete at begin :
2.delete at last :
3.delete at spec :
```

Enter Yout Choice : 1

```
1.Insert an Element :  
2.Delete An Element :  
3.Traverse List :  
4.Search Element :  
5.Count of Element :  
6.Reverse Traverse :  
7.Exit :
```

Enter Your Choice : 2

```
1.delete at begin :  
2.delete at last :  
3.delete at spec :
```

Enter Your Choice : 2

```
1.Insert an Element :  
2.Delete An Element :  
3.Traverse List :  
4.Search Element :  
5.Count of Element :  
6.Reverse Traverse :  
7.Exit :
```

Enter Your Choice : 3

====Content of the List====

20

30

```
1.Insert an Element :  
2.Delete An Element :  
3.Traverse List :  
4.Search Element :  
5.Count of Element :  
6.Reverse Traverse :  
7.Exit :
```

Enter Your Choice : 2

```
1.delete at begin :  
2.delete at last :  
3.delete at spec :
```

Enter Your Choice : 3

Enter the Element to Delete : 30

```
1.Insert an Element :  
2.Delete An Element :  
3.Traverse List :  
4.Search Element :  
5.Count of Element :  
6.Reverse Traverse :  
7.Exit :
```

Enter Your Choice : 3

====Content of the List====

20

```
1.Insert an Element :  
2.Delete An Element :  
3.Traverse List :  
4.Search Element :  
5.Count of Element :  
6.Reverse Traverse :  
7.Exit :
```

Enter Your Choice : 7